



# Chinese Pangolin Optimizer

Fecha: 09 de abril de 2026

Javier Morales Diaz - 21.437.727-6 -- Ingeniería Civil Informática  
Ignacio Layana Silva - 21.307.959-K - - Ingeniería Civil Informática  
Matias Ruiz Flores - 21.384.447-4 -- Ingeniería Civil Informática



# 1. Índice

|                                     |           |
|-------------------------------------|-----------|
| <b>1. Índice</b>                    | <b>2</b>  |
| <b>2. Introducción</b>              | <b>3</b>  |
| <b>3. Metaheurística</b>            | <b>3</b>  |
| <b>3.1 Ecuaciones de movimiento</b> | <b>4</b>  |
| <b>3.2 Pseudocódigo</b>             | <b>5</b>  |
| Componentes claves del pseudocódigo | 9         |
| Resumen de los comportamientos      | 10        |
| <b>4. Referencias</b>               | <b>11</b> |

## 2. Introducción

En el mundo real, resolver problemas complejos, ya sea diseñar la red logística más eficiente o crear componentes de ingeniería más ligeros, implica buscar la mejor configuración posible entre millones de opciones. Los métodos matemáticos tradicionales suelen tardar una eternidad en resolver estos desafíos. Para solucionarlo, los investigadores recurren a las metaheurísticas (algoritmos inteligentes que utilizan atajos inspirados en la naturaleza para encontrar soluciones prácticas, rápidas y eficientes.)

Respecto a eso, el presente informe analiza el *Chinese Pangolin Optimizer* (CPO), un algoritmo de optimización. El atractivo de este modelo radica en que imita la particular forma en que el pangolín chino sobrevive, rastrea y caza a sus presas utilizando su sentido del olfato. Al traducir estos instintos biológicos en ecuaciones, el CPO logra un equilibrio entre la "exploración" a ciegas y la "explotación" minuciosa de los datos, demostrando en diversas pruebas ser una herramienta veloz y robusta para enfrentar los problemas de optimización más exigentes de la actualidad.

## 3. Metaheurística

El CPO es un algoritmo bioinspirado donde la metáfora central se basa en las estrategias de supervivencia y caza de los pangolines chinos en su hábitat natural. Estos animales dependen de un sentido del olfato desarrollado para detectar la presencia de sus presas (principalmente hormigas y termitas). La metaheurística modela esta interacción natural utilizando la concentración del aroma emitido por las presas como un "interruptor" matemático. Dependiendo de si la concentración de este olor es alta o baja, el algoritmo decide de manera inteligente y dinámica qué estrategia de búsqueda debe adoptar para encontrar la fuente de alimento (la solución óptima).

Esta inspiración biológica se traduce en el algoritmo mediante la alternancia de dos comportamientos principales: la Depredación (Predation) y la Atracción (Luring). Cuando la concentración de olor es baja o el pangolín decide cazar activamente, entra en modo de Depredación; en el algoritmo, esto representa la fase de exploración global, donde se realizan movimientos amplios para rastrear el espacio de búsqueda. Por el contrario, cuando la concentración de olor es muy alta (detecta que está rodeado de presas), el pangolín entra en modo de Atracción: finge estar muerto, secreta un olor dulce y deja que las hormigas se acerquen a él. Matemáticamente, esto representa la fase de explotación local, donde el algoritmo deja de hacer grandes saltos y se enfoca en afinar las soluciones cercanas para atrapar la mejor posible.

La transición entre fases no es determinística sino probabilística, controlada por un parámetro adaptativo (la energía del pangolín) que evoluciona conforme avanza la búsqueda. Esto permite que el algoritmo mantenga una exploración significativa en iteraciones tempranas mientras favorece la explotación en etapas tardías.

### 3.1 Ecuaciones de movimiento

La dinámica de movimiento en CPO no es uniforme, sino que se divide en etapas biológicas distribuidas en dos grandes comportamientos, controlados por la concentración de aroma ( $C_m$ ) y factores de energía.

1. Comportamiento de Atracción (Luring): Se activa cuando el pangolín 'juega al muerto' para atraer hormigas. En la etapa de desplazamiento para alimentarse, se integran vuelos de Lévy ( $L_{levy}$ ), que permiten realizar saltos aleatorios de gran alcance en el espacio de búsqueda, garantizando una exploración global robusta.

Cuando hay buen olor a hormigas ( $C_m \geq 0.2$ ) y el factor aleatorio (instinto) le dice al pangolín que es mejor tender una trampa en lugar de salir a cazar.

$$D_A(t) = |a \cdot X_A(t) - X_M(t)|$$
$$X_A(t+1) = X(t) + X_A(t) - A_1 \cdot D_A(t)$$

En lugar de mover la posición principal (el pangolín), la fórmula mueve las posiciones de las presas hacia el pangolín. El algoritmo toma las soluciones secundarias y las "arrastra" hacia la mejor solución actual.

Después de atrapar a las presas atraídas, el pangolín hace un movimiento final para comer.

$$D_M(t) = |C_1 \cdot D_A(t) - X_M(t)| + L_{levy} \cdot \left(1 - \frac{t}{T}\right)$$
$$X_M(t+1) = X(t) + X_M(t) - A_1 \cdot D_M(t)$$

La ecuación introduce un movimiento aleatorio muy pequeño que permite al algoritmo afinar la posición principal sobre las presas atrapadas antes de dar por terminada la ronda. Asegura que no queden errores milimétricos en el cálculo final.

2. Comportamiento de Depredación (Predation): Ocurre cuando el aroma es bajo o el instinto dicta una búsqueda activa. Aquí el algoritmo transita desde una búsqueda errática hacia una aproximación rápida y finalmente a la excavación, donde las ecuaciones convergen con precisión hacia el óptimo local (la fuente de alimento).

Al principio de la caza, cuando el olor a hormigas es nulo o muy débil ( $0 \leq C_m < 0.3$ ). El pangolín sabe que hay comida en la zona, pero no tiene un rastro claro a seguir.

$$D_M(t) = |L_{levy} \cdot X_M(t) - X(t)|$$
$$X_M(t+1) = \sin(C_1 \cdot X(t) + A_1 \cdot |X_M(t) - L_{levy} \cdot D_M(t)|)$$

La fórmula utiliza el "Vuelo de Lévy" y una función seno para forzar a las variables de decisión a dar "saltos locos" y erráticos. El objetivo es explorar regiones completamente nuevas y alejadas del mapa para evitar quedarse atrapado en un óptimo local

Cuando el pangolín de repente detecta un rastro de olor moderado ( $0.3 \leq C_m < 0.6$ ). Ya encontró la pista y sabe en qué dirección general debe moverse.

$$D_M(t) = |a \cdot X_M(t) - X(t)|$$

$$X_M(t+1) = X(t) - A_1 \cdot |X_M(t) - \exp(-a) \cdot \sin(rand \cdot \pi) \cdot D_M(t)|$$

Aquí deja de saltar a ciegas y empezar a "correr" hacia la mejor solución conocida. El algoritmo resta la distancia a la mejor posición, lo que "empuja" rápidamente las variables hacia esa zona prometedora ( $\exp(-a)$  simula el rastro de olor). Conserva una pequeña variación matemática para no ir en línea recta perfecta y poder escanear los alrededores mientras corre (moviendo la cabeza de izquierda a derecha por la función seno).

Cuando el olor a hormigas es fortísimo ( $C_m \geq 0.6$ ). El pangolín está literalmente parado sobre el nido.

$$D_M(t) = |C_1 \cdot X_M(t) - X(t)|$$

$$X(t+1) = X(t) + A_1 \cdot |X_M(t) - D_M(t)|$$

El algoritmo asume que la solución está a una distancia cercana. Las modificaciones a las variables de decisión se vuelven minúsculas. El algoritmo se dedica a ajustar los pequeños decimales de la posición para garantizar que está en el punto exacto más óptimo posible.

### 3. Desglose de las variables:

- $A_1$  = representa el factor de fluctuación de energía de las hormigas.
- $D_a(t)$  = distancia de atracción que existe entre el pangolín y la hormiga.
- $a$  = factor de trayectoria de aroma (el rastro de olor viaja por el aire ).
- $D_m(t)$  = Distancia de movimiento calculada entre la posición actual del pangolín y la mejor posición o un salto aleatorio. Determina qué tan largo será el paso que dará el pangolín.
- $X(t)$  = Mejor posición global.
- $X_m(t)$  = Posición del pangolín.
- $X_a(t)$  = Posición de las presas (hormigas)
- $C_1$  = Factor de disminución rápida.
- $L_{levy}$  = Vuelo de levy (generar números aleatorios que simulan "saltos").
- $r_1$  = número aleatorio de instinto. (para escoger comportamiento).
- $t$  = iteración actual.
- $T$  = iteraciones máximas.
- $rand$  = número aleatorio para dar variaciones naturales de movimiento.

## 3.2 Pseudocódigo

Algoritmo: ChinesePangolinOptimizer

Entrada:

- $f$ : Función objetivo a optimizar
- $\text{bounds}$ : Límites del espacio de búsqueda
- $n$ : Número de pangolines
- $T$ : Número máximo de iteraciones
- $d$ : Dimensiones del problema

Salida:

- $X^*$ : Mejor solución encontrada
- $f(X^*)$ : Fitness de la mejor solución

### Fase 0: Inicialización (Dispersión de pangolines)

ENTRADA:

- $n_{\text{pangolines}}$ : número de agentes en la población
- $T_{\text{max}}$ : número máximo de iteraciones
- $\text{dimensionalidad}$ : número de variables de decisión
- $\text{bounds}$ : límites del espacio de búsqueda  $[L_{\text{min}}, L_{\text{max}}]$

INICIALIZAR:

- Generar población  $X$  de  $n$  pangolines aleatoriamente en  $\text{bounds}$ .
- Evaluar fitness de cada pangolín.
- Identificar  $X_M$  (Mejor solución / Pangolín) y  $X_A$  (Segunda mejor / Hormiga).
- $t \leftarrow 1$

### Fase 1: Ciclo Principal ( $t \leq T_{\text{max}}$ )

PASO 1.1: ACTUALIZAR PARÁMETROS SENSORIALES

- $C_M \leftarrow$  Calcular concentración de **aroma** (basado en convergencia).
- $a \leftarrow 2 * \text{random}() * (1 - t/T_{\text{max}})$  // Trayectoria del aroma decreciente.
- $E \leftarrow 2 * \exp(-t / T_{\text{max}})$  // Energía del pangolín.
- $L_{\text{levy}} \leftarrow$  Generar vuelo de Lévy ( $\beta=1.5$ ).

PASO 1.2: ACTUALIZAR CADA PANGOLÍN (Bucle de Agentes)

PARA cada pangolín  $i$ :

- $A_1 \leftarrow 2 * \text{random}() - 1$  // Fluctuación de energía.
- $r \leftarrow \text{random}()$

```
// --- DECISIÓN SENSORIAL ---
SI (C_M  $\geq$  0.2) Y (r  $\leq$  0.5) ENTONCES

    // COMPORTAMIENTO: ATRACCIÓN (Luring)
    SI r < 0.25 ENTONCES
        // ETAPA 1: Atracción y Captura
        D_A  $\leftarrow$  |a * X_A - X[i]|
        X[i]  $\leftarrow$  X[i] + X_A - A_1 * D_A
    SINO
        // ETAPA 2: Movimiento y Alimentación
        D_M  $\leftarrow$  |C_1 * X_A - X_M| + L_levy * (1 - t/T_max)
        X[i]  $\leftarrow$  X[i] + X_M - A_1 * D_M
    FIN SI

SINO // (C_M  $\leq$  0.7) O (r > 0.5)

    // COMPORTAMIENTO: DEPREDACIÓN (Predation)
    SI C_M < 0.3 ENTONCES
        // ETAPA 3: Búsqueda y Localización (Exploración)
        D_M  $\leftarrow$  |L_levy * X_M - X[i]|
        X[i]  $\leftarrow$  sin(C_1 * X[i] + A_1 * |X_M - L_levy * D_M|)

    SINO SI (0.3  $\leq$  C_M < 0.6) ENTONCES
        // ETAPA 4: Acercamiento Rápido
        D_M  $\leftarrow$  |a * X_M - X[i]|
        X[i]  $\leftarrow$  X[i] - A_1 * |X_M - exp(-a) * sin(r *  $\pi$ ) * D_M|

    SINO // C_M  $\geq$  0.6
        // ETAPA 5: Excavación y Alimentación (Explotación)
        D_M  $\leftarrow$  |C_1 * X_M - X[i]|
        X[i]  $\leftarrow$  X[i] + A_1 * |X_M - D_M|
    FIN SI
FIN SI

X[i]  $\leftarrow$  Controlar_Límites(X[i])
fitness[i]  $\leftarrow$  Evaluar(X[i])
FIN PARA

PASO 1.3: ACTUALIZAR LÍDERES GLOBALES
- Actualizar X_M y X_A con los mejores resultados de la iteración.
- t  $\leftarrow$  t + 1

RETORNAR X_M y fitness(X_M)
```

Retorna:

- $X\_M$ : mejor solución encontrada
- $\text{fitness}(X\_M)$ : valor óptimo

### Componentes claves del pseudocódigo

- **E(Energía)**: Inicia en 2 y decrece exponencialmente hasta 0. Controla la "fuerza" o amplitud base del movimiento del algoritmo a lo largo del tiempo.
- **A\_1 (Fluctuación de Energía)**: Varía aleatoriamente entre -1 y 1 multiplicando a la energía. Determina si el movimiento en esa iteración específica se agranda o se reduce.
- **C\_M (Concentración de Aroma)**: Valor entre 0 y 1 que actúa como interruptor principal. Indica qué tan cerca está la población de la solución óptima y decide el comportamiento a tomar.
- **C\_1 (Factor de Disminución)**: Varía entre -1 y 1. Actúa como un factor de contracción en las ecuaciones matemáticas para enfocar la búsqueda.
- **L\_{levy} (Vuelo de Lévy)**: Operador estocástico que permite al algoritmo realizar saltos grandes e impredecibles, garantizando una exploración global robusta.
- **Fatiga (F)**: Calculada como  $\log(1+t)$ . En la implementación de este pseudocódigo es solo un valor de referencia para monitorear el avance, pero no afecta directamente las ecuaciones de movimiento.

### Resumen de los comportamientos

Atracción / Luring (Cuando la presa está fuertemente detectada):

- **Estrategia**: Se divide en dos etapas (primero atrae a la presa, luego se mueve para capturarla).
- **Dinámica**: Genera un movimiento directo y agresivo hacia un punto focal.
- **Matemática**: Utiliza un desplazamiento vectorial directo, complementado al final con un Vuelo de Lévy atenuado por el tiempo para el ajuste milimétrico.

Depredación / Predation (Con diferentes estrategias según el aroma ( $C\_M$ )):

- **$C\_M$  bajo - BÚSQUEDA**:  
El algoritmo no encuentra un rastro claro de la solución. Por lo tanto, realiza una exploración global con movimientos estocásticos y saltos amplios (sin una dirección fija) para evitar óptimos locales.



- **C\_M medio - APROXIMACIÓN RÁPIDA:**

El algoritmo detecta un rastro prometedor y se acerca de manera modulada. Genera movimientos direccionales hacia la mejor solución conocida, pero manteniendo una leve variación matemática para escanear el área.

- **C\_M alto - EXCAVACIÓN:**

El algoritmo ha localizado el punto óptimo. Transita hacia una fase de explotación pura, deteniendo la exploración y realizando una convergencia intensiva y milimétrica hacia el objetivo final.

**Tabla de Características**

| Característica     | Descripción                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Exploración Global | Reforzada mediante vuelos <b>de Lévy</b> y altas fluctuaciones de energía ( $A_1$ ), permitiendo saltar a regiones inexploradas del espacio.        |
| Explotación Local  | Lograda en la fase de excavación, donde el factor de fatiga y la alta concentración de aroma ( $C_M > 0.6$ ) fuerzan movimientos de alta precisión. |
| Adaptabilidad      | El algoritmo alterna dinámicamente entre atraer o depredar según el éxito de la iteración anterior (retroalimentación olfativa).                    |

## 4. Metaheurística Binaria

Dado que el algoritmo CPO original está diseñado para operar en espacios de búsqueda continuos mediante ecuaciones que utilizan funciones trigonométricas y exponenciales, es necesario aplicar una **técnica de dos pasos** para adaptar el movimiento del pangolín a dominios binarios  $[0,1]$ , propios de problemas combinatorios como el *Set Covering Problem*.

### 4.1. Técnica de dos pasos

La transición del dominio continuo al binario se realiza mediante los siguientes componentes:

1. **Paso 1: Función de Transferencia (T):** Se utiliza para mapear los valores continuos resultantes de las ecuaciones de depredación y atracción a un rango de probabilidad  $[0, 1]$ . Para este algoritmo, se implementa la función **Sigmoide (S-shape)**:

$$T(x_{i,j}^k) = \frac{1}{1 + e^{-x_{i,j}^k}}$$

Donde  $x_{i,j}^k$  representa la posición continua calculada para la dimensión  $j$  del agente  $i$  en la iteración  $k$ .

2. **Paso 2: Regla de Binarización:** Una vez obtenida la probabilidad, se aplica una regla estocástica para determinar el nuevo valor discreto. Se utiliza la **Regla Estándar**:

$$X_{i,j}^{k+1} = \begin{cases} 1 & \text{si } rand < T(x_{i,j}^k) \\ 0 & \text{en otro caso} \end{cases}$$

Donde *rand* es un número aleatorio uniforme en el intervalo  $[0, 1]$ .

### 3.2. Incorporación en el proceso (Pseudocódigo)

La técnica de binarización se incorpora inmediatamente después de la actualización de la posición continua y antes de la evaluación del *fitness*. A continuación se muestra el segmento del ciclo principal modificado

#### # PASO 1.2: ACTUALIZAR CADA PANGOLÍN (Bucle de Agentes)

PARA cada pangolín i:

```
A_1 = 2 * random() - 1  
r = random()
```

```
# --- DECISIÓN SENSORIAL (Predation o Luring) ---
```

```
SI (C_M >= 0.2) Y (r <= 0.5):
```

```
    # ... [Ecuaciones de Atracción] ...
```

```
SINO:
```

```
    # ... [Ecuaciones de Depredación] ...
```

```
FIN SI
```

```
# --- INCORPORACIÓN DE TÉCNICA BINARIA ---
```

```
PARA cada dimensión j:
```

```
    # Paso 1: Transformación a probabilidad (Sigmoide)
```

```
    prob = 1 / (1 + exp(-X[i][j]))
```

```
    # Paso 2: Aplicación de Regla de Binarización
```

```
    SI random() < prob:
```

```
        X[i][j] = 1
```

```
    SINO:
```

```
        X[i][j] = 0
```

```
    FIN SI
```

```
FIN PARA
```

```
# --- VALIDACIÓN Y EVALUACIÓN ---
```

```
X[i] = Reparar_Solución(X[i]) # Asegurar factibilidad del USCP
```

```
fitness[i] = Evaluar(X[i])
```

```
FIN PARA
```



## 5. Referencias

- [1] Guo, Z. Q., Liu, G. W., & Jiang, F. (2025). [Chinese Pangolin Optimizer: a novel bio-inspired metaheuristic for solving optimization problems](#) [Article Number 517]